

Alocação Integrada de Recursos e Seleção de Clientes em Redes de Borda: Um Modelo Multi-objetivo para Aprendizado Federado

RESUMO

O Aprendizado Federado (AF) viabiliza o treinamento colaborativo de modelos de aprendizado de máquina preservando a privacidade dos dados. No entanto, sua implementação em redes com restrições de energia e heterogeneidade impõe desafios. Para lidar com o conflito entre consumo energético, tempo de convergência e desempenho, este trabalho propõe uma formulação de programação inteira mista (PIM) não linear multi-objetivo. A modelagem integra dois problemas fundamentais do AF: a seleção de clientes e a alocação de recursos (frequência de CPU), atuando de forma adaptativa com a evolução das rodadas de treinamento de aprendizado de máquina. O algoritmo evolutivo NSGA-II é utilizado para solucionar o problema. A proposta é validada empiricamente por meio de emulação de rede utilizando o MininetFed. Os resultados demonstram que a metodologia reduz o consumo de energia para cerca de 10% do exigido no treinamento padrão, mantendo a acurácia do modelo global em cenários IID e non-IID.

PALAVRAS CHAVE. Aprendizado Federado, Otimização Multi-objetivo, Seleção de Clientes, Eficiência Energética, NSGA-II.

Otimização Multi-Objetivo (OMO), PO em IA (PO&IA), Simulação (SIM), Otimização não linear (ONL), Meta-heurística (MH)

ABSTRACT

Federated Learning (FL) enables collaborative training of machine learning models while preserving data privacy. However, its implementation in networks with energy constraints and hardware heterogeneity poses challenges. To address the conflict between energy consumption, convergence time, and model performance, this paper proposes a multi-objective nonlinear mixed-integer programming (MINLP) formulation. The model integrates two fundamental FL problems: client selection and resource allocation (CPU frequency), acting adaptively as the machine learning training rounds evolve. The NSGA-II evolutionary algorithm is used to solve the defined problem. The proposal is empirically validated through network emulation using the MininetFed tool. Experimental results show that the methodology reduces energy consumption to about 10% of the value required in the standard training mode, while maintaining the accuracy of the global model in both IID and non-IID data scenarios.

KEYWORDS. Federated Learning, Multi-objective Optimization, Client Selection, Energy Efficiency, NSGA-II,

Multi-Objective Optimization (MOO), Operations Research in AI (OR&AI), Simulation (SIM), Nonlinear Optimization (NLO), Metaheuristics (MH).

1. Introdução

O Aprendizado Federado (AF) revolucionou o treinamento de modelos de aprendizado de máquina em redes distribuídas ao viabilizar a colaboração entre nós descentralizados sem exigir o compartilhamento de dados brutos e sensíveis, lidando com preocupações fundamentais de privacidade e segurança da informação [McMahan et al., 2017; Adsure e Devare, 2024]. Apesar de seus benefícios inerentes para a privacidade, o treinamento federado impõe altos custos operacionais de comunicação e computação. Revisões recentes da literatura destacam algumas barreiras de implementação do AF, dentre elas a heterogeneidade dos dispositivos, manifestada pelo poder computacional variável e disponibilidade energética limitada dos dispositivos, como um fator que pode comprometer severamente a sustentabilidade e a eficiência do treinamento [Yang et al., 2021; Zhou et al., 2023; Guendouzi et al., 2023].

Como resposta a esses desafios, a formulação de técnicas e modelos de otimização no Aprendizado Federado tem ganhado destaque na literatura científica, visando garantir a generalização e eficiência do aprendizado mesmo sob limitações severas de hardware [Adsure e Devare, 2024]. Nesse contexto, o problema de seleção de clientes tornou-se um mecanismo bem conhecido e essencial na literatura. Estratégias têm sido propostas para elencar participantes em cada rodada de treinamento de aprendizado de máquina distribuído, a fim de contornar problemas de ociosidade, falhas de conectividade e mitigar o tempo de treinamento [Nishio e Yonetani, 2019; Tahir e Ali, 2024; Ali et al., 2025]. Paralelamente, o problema de alocação de recursos da rede e do sistema também se consolidou na literatura, englobando desde o controle de largura de banda até o escalonamento da frequência de CPU dos dispositivos para garantir a viabilidade energética do AF [Tran et al., 2019; Kim et al., 2022]. Embora trabalhos recentes tentem orquestrar essas decisões em ambientes de *Edge Computing* [Luo et al., 2020; Wei et al., 2024], gerenciar simultaneamente quem participa e como os recursos são consumidos em cada etapa do processo continua sendo um desafio em aberto.

Mais recentemente, alguns autores passaram a investigar a união dessas estratégias para balancear trade-offs no treinamento [Luo et al., 2020; Wei et al., 2024]. É importante ressaltar que a modelagem matemática apresentada neste artigo é fortemente embasada nos fundamentos que a literatura já estabeleceu para a seleção de clientes e a alocação de recursos em AF. Contudo, para contextualizar e diferenciar este trabalho em relação às abordagens existentes, destaca-se que o problema de otimização aqui definido e estruturado é de autoria própria. Propomos um modelo dinâmico projetado para lidar com o conflito de interesses entre consumo energético, tempo e desempenho (acurácia) ao longo das rodadas de treinamento.

Embora o desafio de conciliar a seleção de clientes e a alocação de recursos não seja um problema novo na literatura de Aprendizado Federado, a maneira de modelá-lo matematicamente para superar as restrições ainda é passível de inovações. Nesse sentido, partindo dos fundamentos já estabelecidos por pesquisas anteriores, este artigo introduz uma formulação inédita, denominada Aprendizado Federado Evolutivo Adaptivo (AFEA). Portanto, o problema consiste em dada uma rede de aprendizado federado com dispositivos heterogêneos, definir a cada rodada quais dispositivos participarão do treinamento, e para os que participarão, como otimizar o consumo de energia mantendo a convergência do modelo em tempo viável. É necessário que essa tomada de decisão se adapte ao cenário de cada rodada, considerando a evolução das métricas do modelo de IA e consequências da seleção dos dispositivos.

Nesse contexto, as principais contribuições deste trabalho são sumarizadas como: i) Uma formulação de programação inteira mista (PIM) não linear multi-objetivo para Aprendizado Federado (AF) adaptativo à evolução das rodadas de treinamento; ii) Modelagem de dois problemas do aprendizado federado de forma integrada: a seleção de clientes e a alocação de recursos, por

meio de um PIM; iii) Frente a formulação PIM apresentada, implementação do algoritmo NSGA-II para solução do problema definido por ela, criando assim a metodologia AFEA; e iv) Validação dos resultados teóricos a partir de uma simulação do treinamento em rede, sensível ao consumo de energia.

O restante deste artigo está organizado da seguinte forma: A [Seção 2](#) apresenta a modelagem e formulação do problema. A [Seção 3](#) detalha a heurística NSGA-II proposta para a solução. A [Seção 4](#) discute os experimentos e resultados obtidos. Por fim, a [Seção 5](#) conclui o trabalho e aponta direções futuras.

2. Modelagem e Formulação do Problema de Aprendizado Federado

Nesta seção são apresentadas a modelagem do aprendizado e a formulação do problema proposto de otimização associado. Toda a notação matemática e símbolos definidos são descritos na Tabela 1.

Considerando uma rede de computadores com topologia arbitrária, a arquitetura do AF proposta neste trabalho estabelece que uma dessas máquinas assume o papel de Servidor de Agregação (SA), enquanto as demais são definidas como Dispositivos Clientes (DC) e compõem o conjunto $\mathcal{DC} = \{DC_1, DC_2, \dots, DC_N\}$. Assume-se que nessa rede, todo DC_i possa se comunicar direta ou indiretamente com SA e vice-versa.

Cada dispositivo DC_i possui seu próprio conjunto de dados $\mathcal{D}_i = \{\{x_1, y_1\}, \{x_2, y_2\}, \dots, \{x_{d_i}, y_{d_i}\}\}$, composto de d_i amostras de pares entrada e saída $\{x_j, y_j\}$, mantido estritamente local e sob nenhuma circunstância, voluntariamente compartilhado com outro dispositivo, embora em certos casos possam existir cópias da mesma amostra em dispositivos diferentes. O tamanho em bytes, de cada *dataset* $\mathcal{D}_i$ é definido por S_i .

Os dispositivos do conjunto $\mathcal{DC}$ possuem a tarefa de executar algum algoritmo de aprendizado de máquina. Nesse trabalho adotamos como exemplo um problema de classificação, e para tanto, cada DC possui sua rede neural de pesos w_i , calibrados para resolver o [Problema 1](#) (definido conforme [Equação \(1\)](#)) de minimização da função de perda $L(w_i) = \frac{1}{d_i} \sum_{j=1}^{d_i} l(x_j, y_j, w_i)$ via SGD (*stochastic gradient descent*) [[Goodfellow et al., 2016](#)]:

$$\text{Problema 1 : } w^* = \arg \min L(w_i). \quad (1)$$

Por outro lado, o SA deve agregar as redes neurais e redistribuir para os dispositivos. A forma como o servidor agrega os pesos pode variar, adotaremos FedAvg [[McMahan et al., 2017](#)], um algoritmo padrão da literatura.

O [Algoritmo 1](#) aborda o funcionamento padrão do Aprendizado Federado. Como entrada, o algoritmo recebe o número total de rodadas R definido para o treinamento dos modelos de aprendizado residentes nos dispositivos. Entretanto é possível considerar também como entrada critérios associados ao desempenho e convergência dos modelos, como acurácia média alvo, F1-score, entre outros. Como saída, o algoritmo retorna o modelo global final w^R . O algoritmo evolui pelas etapas descritas pelas linhas 1 a 12. Na linha 1, SA inicializa o sistema transmitindo os pesos do modelo global inicial w^0 : uma matriz de dimensão $n \times m$ com valores arbitrários (sem perda de generalidade) são transmitidos para todos os DC_i . O *loop* principal do treinamento federado (linhas 2 a 10) é executado por R vezes (da rodada $t = 0$ até $t = R - 1$). Na linha 3, o servidor SA define quais clientes participarão da rodada de treinamento corrente e define aspectos do treinamento para otimizar o consumo de tempo de convergência e de energia. O *loop* das linhas 4 a 6 consiste na fase de Treinamento Local. Neste *loop*, cada dispositivo cliente selecionado ajusta seus novos pesos do modelo de aprendizado w_i^{t+1} resolvendo seu problema de otimização local ([Problema 1](#)) usando o método SGD durante um determinado número de épocas (iterações sobre todo o *dataset*). Após o

Algoritmo 1: Esquema de Aprendizado Federado Distribuído

Entrada: Número R de rodadas
Saída: Modelo global final w^R

```

1 SA inicializa e distribui  $w^0$  para todos os dispositivos  $DC_i, i = 1, \dots, N$ ;
2 for  $t = 0$  to  $(R - 1)$  do
    // Fase de Coordenação e Otimização
3   SA obtém as decisões da rodada: dispositivos selecionados e otimização de tempo e energia  $\leftarrow$ 
    Solução do problema de otimização associado obtida via algoritmo heurístico;
    // Fase de Treinamento Local nos DCs
4   for cada dispositivo cliente  $DC_i$  selecionado do
5     obtém  $w_i^{t+1}$  resolvendo o Problema 1 para  $L(w_i^t)$  via SGD;
6     envia  $w_i^{t+1}$  para SA;
7   end for
    // Fase de Agregação no Servidor
8   SA agrega os pesos conforme FedAvg;
9    $w^{t+1} \leftarrow \frac{1}{d} \sum_{n=1}^N d_i \cdot w_i^{t+1}$ ;
10  SA distribui o novo modelo global  $w^{t+1}$  para os dispositivos;
11 end for
12 return  $w^R$ ;

```

cálculo, os pesos atualizados são enviados de volta ao servidor SA. As linhas 8-10 consistem da Fase de Agregação: ao receber as atualizações locais, o servidor as agrega via FedAvg, gerando o novo modelo global w^{t+1} . Este modelo global atualizado é então distribuído aos dispositivos clientes, preparando a rede para a próxima rodada de treinamento.

Neste trabalho, propomos a formulação AFEA para modelagem do problema de otimização 2 (Seção 2.3) resolvido pelo algoritmo heurístico NSGA-II (Seção 3), visando promover a atualização dinâmica e adaptativa das decisões tomadas na linha 3 do Algoritmo 1. Em vez de parâmetros estáticos definidos a priori, a seleção de clientes e a alocação de recursos são recalculadas a cada rodada considerando o histórico de seleção e a acurácia alcançada pela rede na rodada anterior. Isso permite que o sistema reaja às variações e se adapte ao estado de convergência e ao consumo energético acumulado.

2.1. Limites de Convergência do modelo

Para estimar os limites de convergência de aprendizado, duas considerações quanto à função de perda $l(x_j, y_j, w_i^t)$ da Equação (1) são importantes. A função é ζ -Smoothness e γ -strongly convex [Yang et al., 2021]. A partir da premissa de convexidade e suavidade, podemos estimar que para alcançar uma acurácia local θ , o número mínimo de iterações locais (épocas) $\Psi(\theta)$ necessárias pode ser dado pela Equação (2), enquanto que nessa mesma configuração, o limite inferior $g(\theta)$ do número de rodadas globais R necessárias é definido conforme Equação (3) [Tran et al., 2019]¹.

$$\Psi(\theta) = -\log(1 - \theta), \quad (2)$$

$$g(\theta) = -\frac{\log(1 - \epsilon_0)}{\theta}. \quad (3)$$

O tempo global faltante para atingir a acurácia global ϵ_0 , assumindo que cada rodada leve um tempo T , pode ser portanto estimado como apresentado na Equação (4).

$$T = G(\theta) \cdot T, \quad G(\theta) = \max \{g(\theta_i), \quad i \in \{1, 2, \dots, N\}\}. \quad (4)$$

¹Definimos $g(\cdot)$ como função apenas de θ por que ϵ_0 é assumido como constante.

$$\begin{aligned}
 \min_{f_i^t, \beta_i^t, \psi_i^t, \theta_i^t} \quad & \sum_{n=1}^N \beta_i^t \psi_i^t G(\theta_i^t) \cdot \frac{\alpha_i}{2} c_i S_i (f_i^t)^2, & (7a) \\
 \min_{\theta_i^t, T^t} \quad & G(\theta_i^t) \cdot T^t, & (7b) \\
 \max_{\beta_i^t} \quad & \sum_{n=1}^N \beta_i^t \left[1 + \frac{S_i}{S} \sum_{h=1}^{t-1} (1 - \beta_i^h) \right], & (7c) \\
 \text{sujeito a:} \quad & f_i^{\min} \leq f_i^t \leq f_i^{\max} \quad \forall i, & (7d) \\
 & \frac{\psi_i^t c_i S_i}{f_i^t} \leq T^t \quad \forall i, & (7e) \\
 & \psi_i^t \geq \Psi(\theta_i^t) \quad \forall i, & (7f) \\
 & \sum_{i=1}^N \beta_i \geq 1 \quad \forall i, & (7g) \\
 & \beta_i^t \theta_i^t \geq \beta_i^t (m \cdot \theta_i^{t-1}) \quad \forall i, & (7h) \\
 & \beta_i^t \in \{0, 1\}, \psi_i^t \in \mathbb{N}^* \quad \forall i, & (7i) \\
 & f_i^t, T^t \in \mathbb{R}, 0 \leq \theta_i^t \in \mathbb{R} \leq 1 \quad \forall i. & (7j)
 \end{aligned}$$

Figura 1: Problema 2.

2.2. Consumo e Tempo computacional

Ao realizar o treinamento, os clientes precisam exercer processamento que exige energia e tempo. Para isso assumimos que cada DC_i possui um poder computacional definido por suas frequências mínimas e máximas de CPU, respectivamente f_{min} e f_{max} . Portanto, a energia que o dispositivo DC_i consome numa rodada de treinamento t é dada pela [Equação \(5\)](#), enquanto que o tempo da rodada para que um número ψ_n^t de épocas seja executado, é dado pela [Equação \(6\)](#):

$$E_i^{cmp} = \psi_i^t \frac{\alpha_i}{2} c_i S_i (f_i^t)^2, \quad (5)$$

$$T_i^{cmp} = \psi_i^t \frac{c_i S_i}{f_i^t}, \quad (6)$$

onde f_i^t é a frequência de operação durante o treinamento da rodada t , c_i uma constante de ciclos de CPU por bit do *dataset* (obtida experimentalmente) e α_i a capacitância efetiva do *chipset* do cliente. Essa formulação é amplamente replicada na literatura [[Ali et al., 2025](#); [Kim et al., 2022](#); [Tran et al., 2019](#); [Wei et al., 2024](#)].

2.3. Definição do Problema

Com o objetivo de conciliar a seleção de clientes e a alocação de recursos computacionais (tempo de treinamento e consumo), formula-se a seguir um PIM multi-objetivo ([Problema 2](#)). A modelagem proposta visa minimizar o consumo total de energia e o tempo global de treinamento, enquanto maximiza a rotatividade e participação dos nós. As variáveis de decisão associadas a este problema são: f_i^t , β_i^t , ψ_i^t , θ_i^t e T^t (descritas na [Tabela 1](#)). Assumem-se como constantes de entrada α_i , S_i , f_{min_i} , f_{max_i} e c_i , além da acurácia mensurada na rodada anterior θ_i^{t-1} e o histórico de seleção de clientes $\beta_i^h, \forall h < t$.

Símbolo	Descrição	Símbolo	Descrição
$\mathcal{DC}$	Conjunto de dispositivos clientes	T^t	Tempo para a rodada t
DC_i	Dispositivo cliente i	β_i^t	Indicador de participação do dispositivo i
N	Número total de dispositivos clientes	θ_i^t	Acurácia local do dispositivo i
$\mathcal{D}_i$	Conjunto de dados do dispositivo i	ϵ_0	Acurácia global alvo
d_i	Número de amostras no dispositivo i	$\Psi(\theta)$	Iterações locais necessárias
(x_j, y_j)	Par entrada-saída da amostra j	$G(\theta)$	Rodadas globais necessárias
S_i	Tamanho em <i>bytes</i> do conjunto de dados do dispositivo i	f_i^t	Frequência de CPU do dispositivo i
w_i	Vetor de pesos do modelo local	$f_i^{\min}, f_i^{\max}$	Frequências mínima e máxima
t	Índice da rodada global	c_i	Ciclos de CPU por bit
w^t	Modelo global na rodada t	α_i	Capacitância efetiva do dispositivo
w_i^t	Modelo local do dispositivo i na rodada t	ψ_i^t	Número de épocas locais
$l(x_j, y_j, w_i)$	Função de perda por amostra	E_i^{cmp}	Energia consumida
$L(w_i)$	Função de perda média local	T_i^{cmp}	Tempo de computação

Tabela 1: Tabela de Notação.

Portanto, a solução do [Problema 2](#) de otimização é utilizado na linha 3 do [Algoritmo 1](#), ao definir os valores β_i^t que ditam os dispositivos selecionados. Assim como o laço da linha 4 do mesmo algoritmo é executado por cada dispositivo com frequência de CPU f_i^t e critérios de parada $\psi_i^t, \theta_i^t, T^t$. A [Figura 1](#) apresenta a formulação matemática proposta para o [Problema 2](#).

As Equações (7a) e (7b) correspondem respectivamente ao consumo total de energia e tempo total de treinamento que ainda são necessários a partir da rodada t , e que se quer minimizar.

A [Equação \(7c\)](#) por sua vez procura maximizar o número de dispositivos que participarão da rodada. O somatório dentro dos colchetes busca recompensar a seleção de clientes que historicamente não foram selecionados, recompensa essa ponderada pela fração de dados que esses dispositivos possuem em relação a toda a rede de AF. A adoção de uma seleção rotativa é importante, uma vez que a não seleção de um dispositivo pode levar à ausência de certas classes (*labels*) no processo de aprendizado da rede [[Tahir e Ali, 2024](#)].

A [Equação \(7d\)](#) impõe os limites do poder computacional de cada DC. As Equações (7e) e (7f) garantem que nenhum dispositivo levará mais tempo no treinamento do que o determinado para a rodada, e que o número mínimo de épocas será executado para atingir a acurácia alvo dessa rodada. Essas restrições também permitem que dispositivos selecionados tenham tempo de treinamento iguais, mas com números diferentes de épocas ou frequência de operação, o que permite reduzir o consumo de alguns dispositivos sem impactar os demais objetivos.

A [Equação \(7g\)](#) garante que ao menos um dos dispositivos seja selecionado na rodada t , garantindo que ela exista. A [Equação \(7h\)](#) garante a melhoria do modelo local ao passar das rodadas. O multiplicador m , $0, 0 < m < 1, 0$ está presente como uma "folga", para que o valor da acurácia não assuma valores superestimados. Nos experimentos, o valor de m foi ajustado para 0,98. Por fim as [Equações \(7i\) e \(7j\)](#) definem os domínios para as variáveis de escolha.

3. Solução Heurística do Problema

Para solucionar o [Problema 2](#) definido pela formulação apresentada na [Seção 2](#), é utilizado *Non-dominated Sorting Genetic Algorithm II* (NSGA-II) [[Deb et al., 2002](#)]. Esse algoritmo evolutivo é reconhecido por sua eficiência computacional e capacidade de encontrar um conjunto diversificado de soluções na Fronteira de Pareto, do problema multi-objetivo. O [Algoritmo 2](#) ilustra o processo evolutivo adotado pelo NSGA-II para lidar com os domínios matemáticos específicos exigidos pelos limites de operação e restrições temporais da rede.

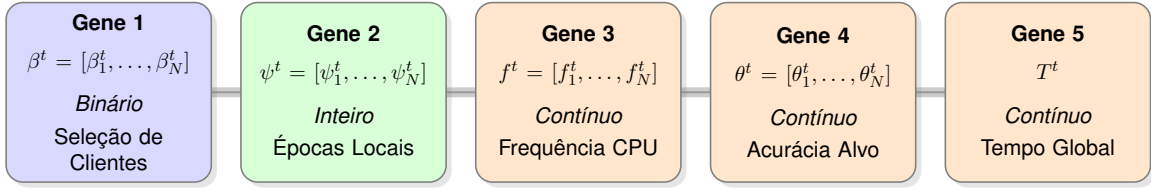


Figura 2: Representação estrutural do cromossomo.

Para alinhar a formulação matemática do [Problema 2](#) à heurística adotada para sua solução, estabelece-se a seguinte representação: cada gene de um cromossomo (ou indivíduo) corresponde a uma variável de decisão, enquanto o cromossomo como um todo codifica uma solução candidata completa para a rodada t , representada pelo vetor $v^t = [\beta^t, \psi^t, f^t, \theta^t, T^t]$, conforme a tipagem heterogênea ilustrada na [Figura 2](#). O processo evolutivo manipula uma população de indivíduos válidos, cujos genes satisfazem as restrições do [Problema 2](#), e gera iterativamente uma prole de novas soluções através de operadores genéticos. O objetivo final do algoritmo é convergir para uma Fronteira de Pareto composta por soluções não-dominadas, ou seja, configurações de rede cujos trade-offs entre energia, tempo e participação não podem ser simultaneamente melhorados.

O processo evolutivo NSGA-II adotado é apresentado no [Algoritmo 2](#). O processo inicia-se com a geração de uma população inicial P_0 composta por indivíduos aleatórios (linha 1). Após avaliar o desempenho e a viabilidade de cada indivíduo frente às restrições do modelo [Problema 2](#) (linha 2), a população é avaliada sob dois critérios essenciais do algoritmo (linha 3): *i*) Rank de não-dominância: classifica a população em múltiplas frentes de Pareto. O rank 1 contém as melhores soluções que não são dominadas por nenhuma outra na população atual, o rank 2 contém as soluções dominadas apenas pelas do rank 1, e assim sucessivamente; *ii*) Crowding distance: métrica que estima a densidade de soluções ao redor de um indivíduo, com finalidade de preservar a diversidade genética da população ao escolher cromossomos que representem indivíduos em perímetros diferentes da região de busca.

Algoritmo 2: Processo Evolutivo NSGA-II para uma Rodada t do [Algoritmo 1](#)

Entrada: gerações G_{max} , acurácia da rodada anterior θ_i^{t-1} , histórico de seleção β_i^h , $h = 0, 1, \dots, t-1$

Saída: Solução v_{best}^t

- 1 Inicializar população P_0 com vetores aleatórios e válidos de $\beta^t, \psi^t, f^t, \theta^t, T^t$;
 - 2 Avaliar objetivos e restrições para P_0 conforme [Problema 2](#);
 - 3 Atribuir rank de não-dominância e *crowding distance* para P_0 ;
 - 4 **for** geração $g = 1$ **to** G_{max} **do**
 - 5 Selecionar indivíduos pais de P_{g-1} via Torneio Binário;
 - 6 Gerar prole Q_g aplicando SBX e Crossover Uniforme nos pais selecionados;
 - 7 Aplicar Mutação Polinomial e Bitflip na prole Q_g ;
 - 8 Avaliar objetivos e restrições para a nova prole Q_g ;
 - 9 $R_g \leftarrow P_{g-1} \cup Q_g$;
 - 10 $P_g \leftarrow$ Selecionar os N_{pop} melhores indivíduos de R_g via rank e *crowding distance*;
 - 11 **end for**
 - 12 Identificar a Fronteira de Pareto na população final $P_{G_{max}}$;
 - 13 $v_{best}^t \leftarrow$ Selecionar solução da Fronteira utilizando o método de Pseudo-Pesos;
 - 14 **return** v_{best}^t ;
-

O laço principal das linhas 4 a 12 itera sobre as gerações. A cada geração, a seleção dos indivíduos que atuarão como pais da próxima prole é feita via Torneio Binário a partir da população anterior (linha 5). Este operador de seleção sorteia dois indivíduos aleatoriamente e elege o vencedor baseado no critério de dominância, e em caso de empate (mesma frente de Pareto), aqueles situados em regiões menos populosas (maior *crowding distance*).

Os operadores genéticos de cruzamento e mutação são aplicados sobre os pais selecio-

nados para gerar a prole Q_g (linhas 6 e 7). Respeitando os domínios matemáticos das variáveis (conforme a Figura 2), aplicam-se diferentes operadores de acordo com a tipagem de cada gene: i) Para genes contínuos e inteiros ($f^t, \theta^t, T^t, \psi^t$): Aplica-se inicialmente o *Simulated Binary Crossover* (SBX), um operador que cria filhos próximos dos pais ao combinar as coordenadas dos seus genes. Em seguida, emprega-se a Mutação Polinomial que aplica uma pequena perturbação a um gene real segundo uma distribuição polinomial parametrizada por um índice de mutação, que regula a probabilidade de grandes variações versus pequenas variações [Deb et al., 1995]; ii) Para genes binários (β^t): Utiliza-se o *Crossover Uniforme*, no qual cada bit tem probabilidade independente de 50% de ser herdado de qualquer um dos progenitores. Como passo final, aplica-se a mutação *Bitflip*, que inverte o estado lógico do bit mediante uma probabilidade predefinida [Deb et al., 2002].

Após aplicar as variações, o algoritmo avalia os objetivos e restrições dessa nova prole gerada (linha 8). Para garantir o elitismo da evolução, a população original e a prole recém-gerada são unidas em uma matriz intermediária R_g (linha 9). Dessa união conjunta, os N_{pop} melhores indivíduos sobrevivem para formar a próxima geração P_g (linha 10).

Ao final das G_{max} gerações, o algoritmo identifica e consolida uma Fronteira de Pareto repleta de soluções não-dominadas (linha 12), e portanto estão todas empatadas quanto a otimalidade, uma vez que uma solução pode ser superior a outra quanto à um dos objetivos do Problema 2, mas não a todos simultaneamente. Para romper esse empate, implementou-se o método de Pseudo-Pesos para isolar a solução final v_{best}^t (linha 13). Esse método normaliza os valores dos objetivos na Fronteira de Pareto e permite que o decisor atribua um peso de importância para cada objetivo. Por exemplo, ao atribuir o vetor de pesos [0.3, 0.4, 0.3] (Energia, Participação e Tempo, respectivamente), o método calcula a menor distância ponderada entre as soluções da fronteira e o ponto ideal teórico, retornando a solução que guiará a rodada de treinamento (linha 14).

4. Experimentos

Para validação da metodologia proposta, uma rede de AF foi emulada com a ferramenta MininetFed [Sarmiento et al., 2024]. Nesses experimentos os dispositivos virtuais são emulados por containers Docker, e possuem poder computacional distinto. O consumo de energia foi medido pelo modelo de potência baseado em frequência de CPU, assim como de De Vogeleer et al. [2014] e Burd e Brodersen [1996]. As especificações dos dispositivos são apresentadas na Tabela 2, o número reduzido de dispositivos se dá por limitações computacionais do *host* dos experimentos, uma vez que o controle de frequência de CPU limita o número de dispositivos ao número de núcleos físicos da máquina.

CD_n	S_n (MB)	c_n	f_n^{max} (GHz)	f_n^{min} (GHz)
CD_1	32.5	9	2.3	1.3
CD_2	54.1	5	2.8	1.3
CD_3	81.2	4	2.7	1.3
CD_4	43.3	8	2.1	1.3
CD_5	65.0	4	2.5	1.3
CD_6	21.6	11	2.1	1.3

Tabela 2: Configuração dos DCs candidatos ao treinamento.

A constante α_i foi fixada em $0,37 \cdot 10^{-28}$ para todos os dispositivos, uma vez que seu valor depende intrinsecamente do hardware que hospeda a simulação. Tal escolha não compromete a generalidade do modelo, pois as constantes c_i são calibradas individualmente para cada dispositivo e, em conjunto com os respectivos limites de CPU, garantem a heterogeneidade do conjunto de clientes. O critério de parada para o AF (Algoritmo 1) foi definido como 60 rodadas de treinamento, que consiste no número de rodadas em que a evolução do modelo se estabiliza.

A implementação do algoritmo evolutivo NSGAI foi realizada em Python, utilizando a biblioteca *pymoo* [Blank e Deb, 2020]. O algoritmo foi configurado com uma população de 150 indivíduos e 200 gerações. Ao final das gerações, a solução escolhida foi determinada pelo método de Pseudo-Pesos, atribuindo os pesos $[0.3, 0.4, 0.3]$ respectivamente para os objetivos das Equações (7a) a (7c).

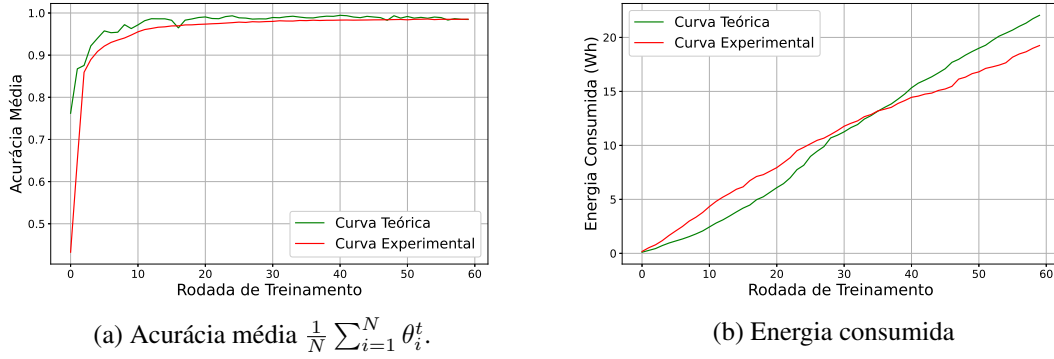


Figura 3: Previsão do modelo para convergência e consumo. Curva Teórica: gerada pelas previsões da formulação. Curva Experimental: resultados empíricos da emulação da rede.

A Figura 3 traz o resultado de 60 rodadas subsequentes do Algoritmo 2 para solução do Problema 2. São avaliados os resultados em termos de acurácia média (Figura 3a) e energia consumida (Figura 3b). As denominações Curva Teórica e Curva Experimental significam, respectivamente, as curvas geradas pelas previsões da formulação e pelos resultados empíricos da emulação da rede. Para a "Curva teórica", a entrada de θ_i^{t-1} no Problema 2 é o valor da solução encontrada na rodada anterior, assim como o valor de energia no gráfico da Figura 3b é dado pela Equação (7a). Por outro lado, a "Curva Experimental" possui dados provenientes da ferramenta de emulação, com o valor de entrada θ_i^{t-1} dado pela medida empírica de cada dispositivo na rodada $t - 1$.

4.1. Resultados de Validação

A proximidade entre o comportamento das curvas teóricas e experimentais da Figura 3 traz evidências de que a formulação proposta é capaz de estimar consumo e tempo de convergência antes que o treinamento seja realizado.

A AFEA é comparada com outras duas estratégias: **i)** LeastEnergy: estratégia de seleção proposta por Bastos et al. [2025], nativa ao MininetFed. **ii)** Padrão: em que todos os dispositivos são selecionados. Para o treinamento dos clientes recebem amostras do dataset MNIST [LeCun et al., 2010], em que são analisados dois casos de distribuição de dados entre os clientes: *i)* IID: todos os dispositivos possuem amostras para todas as labels do dataset (para o MNIST, algarismos de 0 a 9 escritos a mão); *ii)* non-IID: cada dispositivo recebe amostras de apenas 4 das 10 labels existentes.

A Figura 4 evidencia uma redução no consumo de energia. Por outro lado, a Figura 5 demonstra que a generalização do modelo não foi penalizada, mesmo com tempo reduzido na duração do treinamento. A Tabela 3 apresenta que a acurácia máxima da AFEA no cenário IID igualou-se à do Padrão (0,985). Mesmo no cenário non-IID, onde o risco de perda de classes é crítico, a heurística de rotatividade e recompensa da AFEA reteve a robustez do aprendizado com 0,960 de acurácia, mantendo-se competitiva perante aos 0,994 do método que exaure todos os clientes.

Conforme a Tabela 3, o número médio de dispositivos ativados durante as rodadas se equiparou ao método LeastEnergy. E ainda assim para o caso non-IID, AFEA consumiu apenas

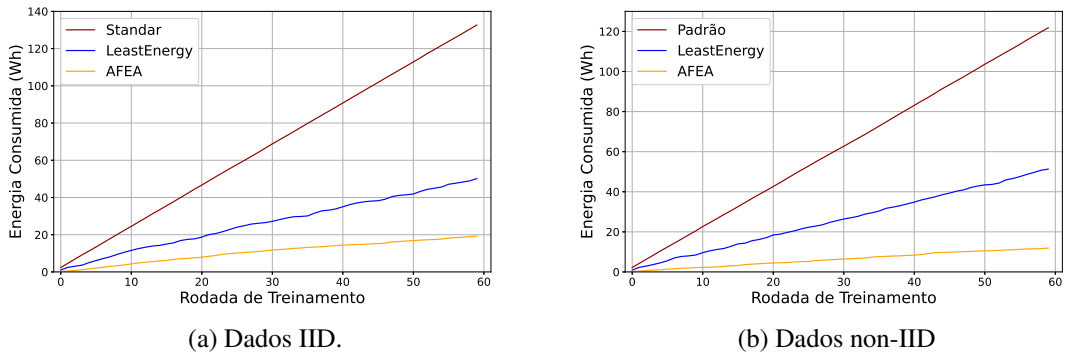


Figura 4: Comportamento do consumo de energia para diferentes cenários.

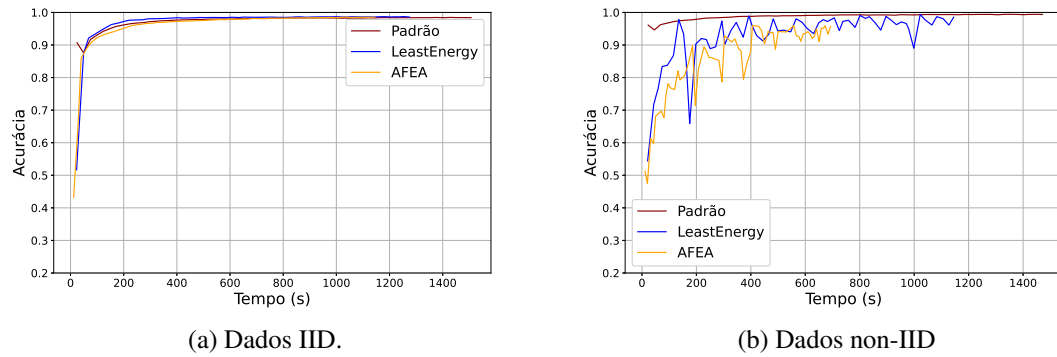


Figura 5: Evolução de acurácia média.

9,0% do que seria consumido no modo padrão de treinamento. Para o caso IID a economia foi menor, sendo 14,5% a razão entre o consumido pela AFEA e Padrão. Quando comparado com LeastEnergy, AFEA consumiu 38,3% para dados IID, e 23,2% para dados non-IID.

5. Conclusão

Este trabalho propôs uma formulação PIM não linear multi-objetivo, que define o problema integrado de seleção de clientes e a alocação de frequências de CPU no AF. A formulação acompanha dinamicamente a evolução do treinamento e sua validação é respaldada pelos resultados experimentais obtidos por meio do emulador MininetFed. O algoritmo evolutivo NSGA-II adaptado para solução do PIM, provou-se uma heurística viável e eficiente.

Os resultados evidenciam que a metodologia AFEA é capaz de reduzir o consumo de energia para cerca de 9,8% do valor exigido no modo padrão de treinamento. Além disso, essa

Metodologia	Consumo (Wh)				Acurácia máxima atingida		Média de Seleccionados por rodada	
	IID	%	non-IID	%	IID	non-IID	IID	non-IID
Padrão	132,679	100	121,792	100	0,985	0,994	6,000	6,000
LeastEnergy	50,199	37,8	51,319	38,7	0,987	0,994	2,967	3,050
AFEA	19,248	14,5	11,920	9,0	0,985	0,960	2,933	2,950

Tabela 3: Comparação de métricas de treinamento emulado no MininetFed.

economia de energia e tempo foi alcançada sem comprometer a acurácia final do modelo global, tanto em cenários de dados IID quanto non-IID. Conclui-se que a gestão inteligente e evolutiva dos recursos computacionais se apresenta como uma alternativa à implementação do Aprendizado Federado em dispositivos com capacidades heterogêneas.

Como trabalhos futuros, sugere-se a aplicação de métodos exatos para encontrar a solução ótima do problema, mesmo em instâncias pequenas como a apresentada. Sugere-se também a avaliação do modelo frente a *datasets* mais desafiadores, além da implementação de outras propostas da literatura no mesmo ambiente de validação, visando comparações entre as metodologias.

Agradecimentos

Este trabalho possui financiamento parcial de CNPq; CAPES (Código de financiamento #001); Fapes (#2026-6NX7B, #2026-B74MN, #2023-RWXSZ); e Fapesp/ MCTI/ CGI.br (Porvir-5G).

Declaração de uso de IA Generativa e tecnologia assisidas por IA no processo de escrita

Durante a preparação deste trabalho, os autores utilizaram o Gemini Pro para aprimorar a linguagem e a legibilidade. Após a revisão com essa ferramenta, os autores editaram o conteúdo conforme necessário e assumem total responsabilidade pelo conteúdo do artigo publicado.

Referências

- Adsure, S. e Devare, M. (2024). A research review on challenges of federated machine learning. In *2024 8th International Conference on Computing, Communication, Control and Automation (ICCUBEA)*, p. 1–7.
- Ali, N., Aloï, G., Gravina, R., Savaglio, C., Sodhro, A. H., e Fortino, G. (2025). Fedheur: Multi-heuristic client selection for task offloading in federated learning. In *2025 21st International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, p. 816–821.
- Bastos, J. J. S., Batista, J. P. C., dos Reis Fontes, R., Cerqueira, E., Villaça, R. S., e Mota, V. F. S. (2025). Otimizando energia no aprendizado federado em redes de baixa potência e com alta taxa de perda de pacotes. In *Anais do XLIII Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos*, p. 43–56, Porto Alegre, RS, Brasil. SBC. URL <https://sol.sbc.org.br/index.php/sbrc/article/view/35121>.
- Blank, J. e Deb, K. (2020). pymoo: Multi-objective optimization in python. *IEEE Access*, 8: 89497–89509.
- Burd, T. D. e Brodersen, R. W. (1996). Processor design for portable systems. *J. VLSI Signal Process. Syst.*, 13(2–3):203–221. ISSN 0922-5773.
- De Vogeleer, K., Memmi, G., Jouvelot, P., e Coelho, F. (2014). The energy/frequency convexity rule: Modeling and experimental validation on mobile devices. In Wyrzykowski, R., Dongarra, J., Karczewski, K., e Waśniewski, J., editors, *Parallel Processing and Applied Mathematics*, p. 793–803, Berlin, Heidelberg. Springer Berlin Heidelberg. ISBN 978-3-642-55224-3.
- Deb, K., Pratap, A., Agarwal, S., e Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197.
- Deb, K., Agrawal, R. B., et al. (1995). Simulated binary crossover for continuous search space. *Complex systems*, 9(2):115–148.

- Goodfellow, I., Bengio, Y., e Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.
- Guendouzi, B. S., Ouchani, S., EL Assaad, H., e EL Zaher, M. (2023). A systematic review of federated learning: Challenges, aggregation methods, and development tools. *Journal of Network and Computer Applications*, 220:103714. ISSN 1084-8045. URL <https://www.sciencedirect.com/science/article/pii/S1084804523001339>.
- Kim, J., Kim, D., Lee, J., e Hwang, J. (2022). A novel joint dataset and computation management scheme for energy-efficient federated learning in mobile edge computing. *IEEE Wireless Communications Letters*, 11(5):898–902.
- LeCun, Y., Cortes, C., e Burges, C. (2010). Mnist handwritten digit database. *ATT Labs [Online]*. Available: <http://yann.lecun.com/exdb/mnist>, 2.
- Luo, S., Chen, X., Wu, Q., Zhou, Z., e Yu, S. (2020). Hfel: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning. *IEEE Transactions on Wireless Communications*, 19(10):6535–6548.
- McMahan, B., Moore, E., Ramage, D., Hampson, S., e Aguera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. In Singh, A. e Zhu, J., editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, p. 1273–1282. PMLR. URL <https://proceedings.mlr.press/v54/mcmahan17a.html>.
- Nishio, T. e Yonetani, R. (2019). Client selection for federated learning with heterogeneous resources in mobile edge. In *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, p. 1–7.
- Sarmiento, E. M., Bastos, J. J., Villaca, R. S., Comarela, G., e Mota, V. F. (2024). Mininetfed: A tool for assessing client selection, aggregation, and security in federated learning. In *2024 IEEE 10th World Forum on Internet of Things (WF-IoT)*, p. 1–6. IEEE.
- Tahir, M. e Ali, M. I. (2024). Multi-criterion client selection for efficient federated learning. *Proceedings of the AAAI Symposium Series*, 3(1):318–322. URL <https://ojs.aaai.org/index.php/AAAI-SS/article/view/31227>.
- Tran, N. H., Bao, W., Zomaya, A., Nguyen, M. N. H., e Hong, C. S. (2019). Federated learning over wireless networks: Optimization model design and analysis. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, p. 1387–1395.
- Wei, X., Ye, K., Shi, X., Xu, C.-Z., e Wang, Y. (2024). Joint participant and learning topology selection for federated learning in edge clouds. *IEEE Transactions on Parallel and Distributed Systems*, 35(8):1456–1468.
- Yang, Z., Chen, M., Saad, W., Hong, C. S., e Shikh-Bahaei, M. (2021). Energy efficient federated learning over wireless communication networks. *IEEE Transactions on Wireless Communications*, 20(3):1935–1949.
- Zhou, Y., Liu, X., e Lei, L. (2023). Multi-objective optimization for bandwidth-limited federated learning in wireless edge systems. *IEEE Open Journal of the Communications Society*, 4:954–966.